GAME PROGRAMMING COURSE — DESIGN PATTERN

シングルトン実践

SINGLE
TON 2

シーン遷移・サウンド・シーン間で共有するデータを作る

対象：クラスまで既習の選抜クラス

前回の振り返り

Lib に入れた2種類のシングルトン

SceneSingleton<T>

シーンを切り替えると**一緒に破棄**される。
見つからなくても**自動では作らない**。
シーン内の UI などを参照してよい。

PersistentSingleton<T>

DontDestroyOnLoad でシーンをまたいで残る。
なければ**自動で作る**（終了中は作らない）。
シーン内オブジェクトの参照は持たない。

POINT 今回は、この2つを**継承したマネージャー**を読みながら、どちらを選ぶべきかを判断できるようにする。

今回のゲーム

ステージを3つクリアするとタイトルへ戻る



- Xxx には **Tutorial / FirstIngame / SecondIngame** が入る（シーン名は **Tutorial_0** など）
- ゴールで **Game Clear** → Enter で次のステージへ。_2 をクリアしたら**クリアを記録**してタイトルへ
- やられたら **Game Over** → Enter で**同じステージをやり直し**
- シーンを切り替えるときは、**いつも画面をフェード**させる

全体の構成

どのクラスが、どちらのシングルトンか

PersistentSingleton　—　起動時に自動生成・ずっと残る

SceneLoadManager
フェードしながらシーンを切り替える

AudioManager
BGM と SE を鳴らす

GameDataHolder
どのステージをクリアしたかを覚えておく

SceneSingleton　—　シーンに置く

InGameUi
Game Over / Game Clear の表示

TPSCameraController
プレイヤーを追う三人称カメラ

POINT　使う側：**Player**（ステートパターンで動く — 次回）と **SceneData**（シーン名を扱う static なクラス・シングルトンではない）

PART 1

SceneLoadManager

1

フェードしながらシーンを切り替える

SceneLoadManager.cs　（フェード付きシーン遷移）

C# Assets/Scripts/SceneLoadManager.cs

```
1  using System.Collections;
2  using UnityEngine;
3  using UnityEngine.UI;
4  using UnityEngine.SceneManagement;
5
6  public class SceneLoadManager : PersistentSingleton<SceneLoadManager>
7  {
8      // フェード用
9      [SerializeField] Image fadeImage;
10     [SerializeField] float fadeTime = 1.0f;
11     [SerializeField] Color fadeColor = Color.black;
12
13     // 他スクリプトからロード中か確認する用
14     public bool isLoading = false;
```

SceneLoadManager.cs （フェード付きシーン遷移）

C# Assets/Scripts/SceneLoadManager.cs

```
16      /// <summary>
17      /// ゲーム起動時に生成
18      /// </summary>
19      [RuntimeInitializeOnLoadMethod]
20      static void Init()
21      {
22          SceneLoadManager pref = Resources.Load<SceneLoadManager>("Singletons/SceneLoadManager");
23          if (pref != null) Instantiate(pref);
24      }
25
26      /// <summary>
27      /// フェードしつつシーンをロードする処理
28      /// </summary>
29      /// <param name="sceneName"></param>
30      public void LoadScene(string sceneName)
31      {
32          if (isLoading) return; // 追加：ロード中なら受け付けない
33          StartCoroutine(_LoadScene(sceneName));
34      }
```

SceneLoadManager.cs （フェード付きシーン遷移）

C# Assets/Scripts/SceneLoadManager.cs

```
36      private IEnumerator _LoadScene(string sceneName)
37      {
38          isLoading = true;
39          Color baseColor = fadeImage.color;
40          float currentTime = 0;
41
42          // フェードアウト
43          while ((currentTime += Time.unscaledDeltaTime) < fadeTime)
44          {
45              float rate = currentTime / fadeTime;
46              Color nextColor = Color.Lerp(baseColor, fadeColor, rate);
47              fadeImage.color = nextColor;
48
49              yield return null;
50          }
51
52          // 非同期でシーンのロード
```

SceneLoadManager.cs　（フェード付きシーン遷移）

C# Assets/Scripts/SceneLoadManager.cs

```
53          var async = SceneManager.LoadSceneAsync(sceneName);
54          while(!async.isDone)
55          {
56              // シーンロード中のフリーズ回避
57              // ロード進捗表示等をする場合はここで
58              yield return null;
59          }
60
61          // フェードイン
62          currentTime = 0;
63          while ((currentTime += Time.unscaledDeltaTime) < fadeTime)
64          {
65              float rate = currentTime / fadeTime;
66              Color nextColor = Color.Lerp(fadeColor, baseColor, rate);
67              fadeImage.color = nextColor;
68
69              yield return null;
70          }
71
72          isLoading = false; // 修正：true → false
73      }
74  }
```

解説 ①

起動時に Resources から自動生成する

C# Assets/Scripts/SceneLoadManager.cs

```
16      /// <summary>
17      /// ゲーム起動時に生成
18      /// </summary>
19      [RuntimeInitializeOnLoadMethod]
20      static void Init()
21      {
22          SceneLoadManager pref = Resources.Load<SceneLoadManager>("Singletons/SceneLoadManager");
23          if (pref != null) Instantiate(pref);
24      }
```

- **RuntimeInitializeOnLoadMethod** → ゲーム起動時に1回だけ呼ばれる
- **Resources.Load** で Assets/Resources/Singletons/ SceneLoadManager.prefab を読む
- Instantiate した瞬間に **Awake** → **DontDestroyOnLoad**

解説 ②

LoadScene はコルーチンを始めるだけ

C# Assets/Scripts/SceneLoadManager.cs

```
30      public void LoadScene(string sceneName)
31      {
32          if (isLoading) return; // 追加：ロード中なら受け付けない
33          StartCoroutine(_LoadScene(sceneName));
34      }
```

ポイント

- **コルーチン：**何フレームにも分けて少しずつ進む処理
- **yield return null** で「続きは次のフレーム」
- ロード中なら受け付けない（**追加した行**）

POINT コルーチンは**動かしている GameObject が消えると止まる**。シーンをまたいで動き続けるには **PersistentSingleton** である必要がある。

解説 ③

フェードアウト ― 少しずつ黒くする

C# Assets/Scripts/SceneLoadManager.cs

```
36      private IEnumerator _LoadScene(string sceneName)
37      {
38          isLoading = true;
39          Color baseColor = fadeImage.color;
40          float currentTime = 0;
41
42          // フェードアウト
43          while ((currentTime += Time.unscaledDeltaTime) < fadeTime)
44          {
45              float rate = currentTime / fadeTime;
46              Color nextColor = Color.Lerp(baseColor, fadeColor, rate);
47              fadeImage.color = nextColor;
48
49              yield return null;
50          }
```

ポイント

- **rate**：0 → 1 に増える割合
- **Color.Lerp(A, B, rate)** A と B の間の色
- **unscaledDeltaTime** → timeScale の影響を受けない

解説 ④

画面が真っ黒なうちにシーンを読み込む

C# Assets/Scripts/SceneLoadManager.cs

```
52        // 非同期でシーンのロード
53        var async = SceneManager.LoadSceneAsync(sceneName);
54        while(!async.isDone)
55        {
56            // シーンロード中のフリーズ回避
57            // ロード進捗表示等をする場合はここで
58            yield return null;
59        }
```

ポイント

- **LoadSceneAsync**：裏で少しずつ読み込む
- **isDone** が true になるまで1フレームずつ待つ
- 待っている間に**進み具合の表示**もできる（async.progress）

解説 ⑤

フェードイン — 元の色に戻す

C# Assets/Scripts/SceneLoadManager.cs

```
61        // フェードイン
62        currentTime = 0;
63        while ((currentTime += Time.unscaledDeltaTime) < fadeTime)
64        {
65            float rate = currentTime / fadeTime;
66            Color nextColor = Color.Lerp(fadeColor, baseColor, rate);
67            fadeImage.color = nextColor;
68
69            yield return null;
70        }
71
72        isLoading = false; // 修正：true → false
73    }
```

ポイント

- 色を**逆向き**に Lerp する
- 最後に **isLoading を false** に戻す

注意

元のコードは最後が **isLoading = true** になっていて、一度ロードすると**ずっとロード中**扱いだった。LoadScene の**連打防止**と合わせて修正している。

もう一歩

自動生成は「いつ」行われるか

書き方	呼ばれるタイミング
[RuntimeInitializeOnLoadMethod] （今回のコード）	AfterSceneLoad：最初のシーンの Awake の後
[RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.BeforeSceneLoad)]	BeforeSceneLoad：最初のシーンを読み込む前

注意

AfterSceneLoad だと、シーンの **Awake で Instance を読んだ瞬間に空っぽの物が自動生成**され、あとから来たプレハブが**重複として消される**。不安なら **BeforeSceneLoad** にしておくと安全。

PART 2

AudioManager

2

BGM と SE を名前で鳴らす

AudioManager.cs （サウンドの再生）

C# Assets/Scripts/AudioManager.cs

```
1  using UnityEngine;
2
3  public class AudioManager : PersistentSingleton<AudioManager>
4  {
5      [Header("Audio Clips")]
6      [SerializeField] private AudioClip[] bgmClips;
7      [SerializeField] private AudioClip[] seClips;
8
9      [Header("Settings")]
10     [SerializeField, Min(1)] private int seSourceCount = 10;
```

AudioManager.cs　（サウンドの再生）

C# Assets/Scripts/AudioManager.cs

```
12      private AudioSource bgmSource;
13      private AudioSource[] seSources;
14
15      /// <summary>
16      /// ゲーム起動時に生成
17      /// </summary>
18      [RuntimeInitializeOnLoadMethod]
19      private static void Init()
20      {
21          AudioManager pref = Resources.Load<AudioManager>("Singletons/AudioManager");
22          if (pref != null) Instantiate(pref);
23      }
24
25      protected override void Awake()
26      {
27          base.Awake();
28
29          // BGM用初期化処理
30          bgmSource = CreateAudioSource("BGM", transform, true);
```

AudioManager.cs　（サウンドの再生）

C# Assets/Scripts/AudioManager.cs

```
32          // SE用初期化処理
33          GameObject seRoot = new GameObject("SE");
34          seRoot.transform.SetParent(transform);
35          seSources = new AudioSource[seSourceCount];
36          for (int i = 0; i < seSources.Length; i++)
37          {
38              seSources[i] = CreateAudioSource($"SE_{i}", seRoot.transform, false);
39          }
40      }
41
42      /// <summary>
43      /// オーディオソースを子オブジェクトに生成
44      /// </summary>
45      /// <param name="objectName">生成名</param>
46      /// <param name="parent">親オブジェクト</param>
47      /// <param name="loop">ループ設定</param>
48      /// <returns></returns>
49      private AudioSource CreateAudioSource(string objectName, Transform parent, bool loop)
50      {
51          GameObject obj = new GameObject(objectName);
52          obj.transform.SetParent(parent);
```

AudioManager.cs （サウンドの再生）

C# Assets/Scripts/AudioManager.cs

```
54          AudioSource source = obj.AddComponent<AudioSource>();
55          source.playOnAwake = false;
56          source.loop = loop;
57
58          return source;
59      }
60
61      /// <summary>
62      /// BGMの再生
63      /// </summary>
64      /// <param name="clipName">BGMの名前</param>
65      public void PlayBGM(string clipName)
66      {
67          // 配列内の同じ名前のクリップを検索
68          AudioClip clip = GetClip(bgmClips, clipName); // 修正：BGMから探す
69
70          // クリップが見つからない場合と、すでに同じのを再生中の場合はキャンセル
71          if (clip == null) return;
72          if (bgmSource.clip == clip && bgmSource.isPlaying) return;
```

AudioManager.cs （サウンドの再生）

C# Assets/Scripts/AudioManager.cs

```
74          bgmSource.clip = clip;
75          bgmSource.Play();
76      }
77
78      /// <summary>
79      /// BGMの停止
80      /// </summary>
81      public void StopBGM()
82      {
83          bgmSource.Stop();
84      }
85
86      /// <summary>
87      /// SEの再生
88      /// </summary>
89      /// <param name="clipName">SEの名前</param>
90      public void PlaySE(string clipName)
91      {
92          // 配列内の同じ名前のクリップを検索
93          AudioClip clip = GetClip(seClips, clipName); // 修正：SEから探す
94          if (clip == null) return;
```

AudioManager.cs （サウンドの再生）

C# Assets/Scripts/AudioManager.cs

```
96          //  現在再生中ではないオーディオソースで再生（ない場合は鳴らさない）
97          foreach (AudioSource source in seSources)
98          {
99              if (source.isPlaying) continue;
100
101             source.clip = clip;
102             source.Play();
103             return;
104         }
105     }
106
107     /// <summary>
108     /// すべてのSEを止める
109     /// </summary>
110     public void StopSE()
111     {
112         foreach (AudioSource source in seSources)
113         {
114             source.Stop();
115         }
116     }
```

AudioManager.cs　（サウンドの再生）

C# Assets/Scripts/AudioManager.cs

```
118     /// <summary>
119     /// 名前検索でクリップを取得
120     /// </summary>
121     /// <param name="clips">探す配列</param>
122     /// <param name="name"></param>
123     /// <returns></returns>
124     private AudioClip GetClip(AudioClip[] clips, string name) // 修正
125     {
126         AudioClip clip = System.Array.Find(clips, x => x.name == name);
127         if (clip == null)
128         {
129             Debug.LogWarning($"音が見つかりません: {name}");
130             return null;
131         }
132         return clip;
133     }
134 }
```

解説 ①

Awake で AudioSource を作る

C# Assets/Scripts/AudioManager.cs

```
25      protected override void Awake()
26      {
27          base.Awake();
28
29          // BGM用初期化処理
30          bgmSource = CreateAudioSource("BGM", transform, true);
31
32          // SE用初期化処理
33          GameObject seRoot = new GameObject("SE");
34          seRoot.transform.SetParent(transform);
35          seSources = new AudioSource[seSourceCount];
36          for (int i = 0; i < seSources.Length; i++)
37          {
38              seSources[i] = CreateAudioSource($"SE_{i}", seRoot.transform, false);
39          }
40      }
```

- 最初に **base.Awake()**（シングルトンの登録）
- BGM 用 1つ、SE 用 **seSourceCount 個**を子オブジェクトとして作る

解説 ②

AudioSource を子オブジェクトに作る

C# Assets/Scripts/AudioManager.cs

```
49      private AudioSource CreateAudioSource(string objectName, Transform parent, bool loop)
50      {
51          GameObject obj = new GameObject(objectName);
52          obj.transform.SetParent(parent);
53
54          AudioSource source = obj.AddComponent<AudioSource>();
55          source.playOnAwake = false;
56          source.loop = loop;
57
58          return source;
59      }
```

- **new GameObject** で空の物を作る
- **SetParent** で子にする
- **AddComponent** で AudioSource を付けて設定

解説 ③

PlayBGM — 同じ曲なら流しっぱなし

C# Assets/Scripts/AudioManager.cs

```
65      public void PlayBGM(string clipName)
66      {
67          // 配列内の同じ名前のクリップを検索
68          AudioClip clip = GetClip(bgmClips, clipName); // 修正：BGMから探す
69
70          // クリップが見つからない場合と、すでに同じのを再生中の場合はキャンセル
71          if (clip == null) return;
72          if (bgmSource.clip == clip && bgmSource.isPlaying) return;
73
74          bgmSource.clip = clip;
75          bgmSource.Play();
76      }
```

ポイント

- 名前で **bgmClips** から探す
- **同じ曲が再生中**なら何もしない
  → リトライしても曲が**最初に戻らない**

解説 ④

PlaySE — 空いている AudioSource で鳴らす

C# Assets/Scripts/AudioManager.cs

```
90      public void PlaySE(string clipName)
91      {
92          // 配列内の同じ名前のクリップを検索
93          AudioClip clip = GetClip(seClips, clipName); // 修正：SEから探す
94          if (clip == null) return;
95
96          //  現在再生中ではないオーディオソースで再生（ない場合は鳴らさない）
97          foreach (AudioSource source in seSources)
98          {
99              if (source.isPlaying) continue;
100
101             source.clip = clip;
102             source.Play();
103             return;
104         }
105     }
```

ポイント

- SE 用 AudioSource を**あらかじめ10個**作っておき、**空いている物**を使い回す
- 全部使用中なら**鳴らさない**
- 「先に作って使い回す」のは**オブジェクトプール**の考え方

解説 ⑤

GetClip — 探す配列を引数で受け取る

C# Assets/Scripts/AudioManager.cs

```
124     private AudioClip GetClip(AudioClip[] clips, string name) // 修正
125     {
126         AudioClip clip = System.Array.Find(clips, x => x.name == name);
127         if (clip == null)
128         {
129             Debug.LogWarning($"音が見つかりません: {name}");
130             return null;
131         }
132         return clip;
133     }
```

ポイント

- **Array.Find**：条件に合う最初の要素
- **x => x.name == name**：ラムダ式（「名前が同じ」という条件）

注意 元のコードは **seClips だけ**を探していたため、PlayBGM で**BGM が見つからなかった**。探す配列を引数で渡すように修正している。